

JavaScript Execution Context

Що таке контекст виконання?

Контекст виконання (Execution Context) — це абстрактне поняття в JavaScript, яке описує середовище, в якому виконується код. Контекст виконання містить інформацію про змінні, функції, аргументи та область видимості, доступні під час виконання коду.

Типи контекстів виконання

У JavaScript існує три типи контекстів виконання:

1. **Глобальний контекст виконання (Global Execution Context)** — створюється під час завантаження скрипта. Це базовий, найзовнішній контекст, у якому виконується код, що не знаходиться всередині функцій.
2. **Контекст виконання функції (Function Execution Context)** — створюється щоразу при виклику функції. Кожна функція має свій власний контекст виконання.
3. **Контекст виконання eval (Eval Execution Context)** — створюється при виконанні коду всередині функції `eval()`.

Стек викликів (Call Stack)

JavaScript використовує структуру даних «стек» для керування контекстами виконання. Це називається «стеком викликів» (Call Stack). Принцип роботи:

- Коли JavaScript починає виконання скрипта, він створює глобальний контекст і поміщає його у стек викликів.
- При виклику функції створюється новий контекст виконання і поміщається на вершину стека.
- Після завершення функції її контекст видаляється зі стека, і виконання повертається до попереднього контексту.

```
function first() {  
  console.log('Inside first function');  
  second();  
  console.log('Again inside first function');  
}  
  
function second() {  
  console.log('Inside second function');  
}  
  
first();  
console.log('Inside Global Execution Context');
```

Порядок створення та видалення контекстів у стеці:

1. Додається глобальний контекст
2. Викликається `first()` — додається контекст `first()`
3. Усередині `first()` викликається `second()` — додається контекст `second()`
4. `second()` завершується — її контекст видаляється зі стека
5. Продовжується виконання `first()` — контекст залишається у стеці
6. `first()` завершується — її контекст видаляється зі стека
7. Продовжується виконання глобального коду

Фази створення контексту виконання

Створення контексту виконання відбувається у дві фази:

1. Фаза створення (Creation Phase)

На цій фазі відбувається:

- Створення об'єкта змінних (Variable Object) / Лексичного оточення (Lexical Environment)
- Створення ланцюжка областей видимості (Scope Chain)
- Визначення значення `this`

Лексичне оточення (Lexical Environment)

Лексичне оточення — це внутрішня структура рушія JavaScript, яка складається з:

- **Environment Record (Запис оточення)** — зберігає оголошення змінних і функцій
- **Посилання на зовнішнє лексичне оточення** — посилання на батьківське оточення

```
// Приклад лексичного оточення (концептуально)
lexicalEnvironment = {
  environmentRecord: {
    // Оголошення змінних і функцій
    // Включаючи аргументи для функціонального контексту
  },
  outer: // посилання на зовнішнє лексичне оточення
}
```

Hoisting (Підняття)

Під час фази створення відбувається явище, яке називається "підняття" (hoisting). Оголошення змінних і функцій переміщуються на початок їхньої області видимості:

- Оголошення функцій піднімаються разом із їхньою реалізацією
- Змінні, оголошені через `var`, піднімаються зі значенням `undefined`
- Змінні, оголошені через `let` і `const`, також піднімаються, але знаходяться у «тимчасовій мертвій зоні» (Temporal Dead Zone) до моменту фактичного оголошення.

```
console.log(x); // undefined (через hoisting)
console.log(y); // ReferenceError: y is not defined
console.log(z); // ReferenceError: Cannot access 'z' before initialization

var x = 10;
let z = 30;

function example() {
  console.log("Це працює завдяки hoisting");
}
```

2. Фаза виконання (Execution Phase)

На цій фазі код виконується рядок за рядком. Змінним присвоюються значення, і викликаються функції.

Контекст this

Значення `this` визначається в момент виклику функції і залежить від того, як функція викликається:

1. **Глобальний контекст:** У глобальному контексті `this` вказує на глобальний об'єкт (у браузері це `window`, у Node.js — `global`).

```
console.log(this); // window у браузері
```

2. **Контекст функції:** У звичайній функції значення `this` залежить від способу виклику:

```
function showThis() {
  console.log(this);
}

showThis(); // window у нестрогому режимі, undefined у строгому режимі

const obj = {
  method: showThis
};

obj.method(); // obj
```

3. **Стрілочні функції:** У стрілочних функціях `this` успадковується з навколишнього лексичного контексту:

```
const obj = {
  regularMethod: function() {
    console.log(this); // obj
  }
}
```

```

    const arrowFunc = () => {
      console.log(this); // obj (успадковується від regularMethod)
    };

    arrowFunc();
  }
};

obj.regularMethod();

```

4. **Явне вказання контексту:** Можна явно вказати значення `this` за допомогою методів `call()`, `apply()` або `bind()`:

```

function greet(greeting) {
  console.log(`${greeting}, я ${this.name}`);
}

const person = { name: 'Іван' };

greet.call(person, 'Привіт'); // Привіт, я Іван
greet.apply(person, ['Доброго дня']); // Доброго дня, я Іван

const boundGreet = greet.bind(person);
boundGreet('Добрий день'); // Добрий день, я Іван

```

Замикання (Closures)

Замикання — це комбінація функції та лексичного оточення, в якому ця функція була оголошена. Завдяки замиканням, функція може мати доступ до змінних із зовнішнього лексичного оточення навіть після завершення виконання зовнішньої функції.

```

function createCounter() {
  let count = 0;

  return function() {
    return ++count;
  };
}

const counter = createCounter();
console.log(counter()); // 1
console.log(counter()); // 2
console.log(counter()); // 3

```

У цьому прикладі внутрішня функція зберігає доступ до змінної `count`, навіть коли контекст виконання функції `createCounter` вже завершився і видалений зі стека викликів.

...